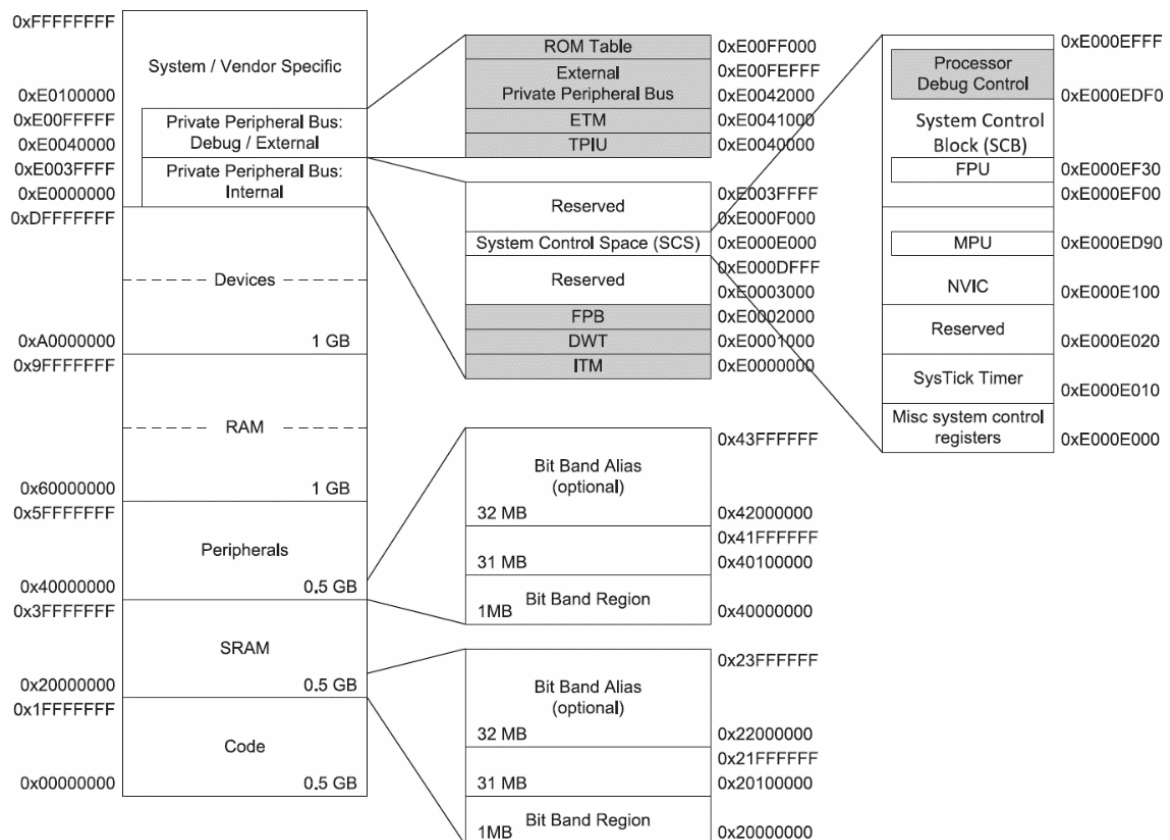


Quick Reference

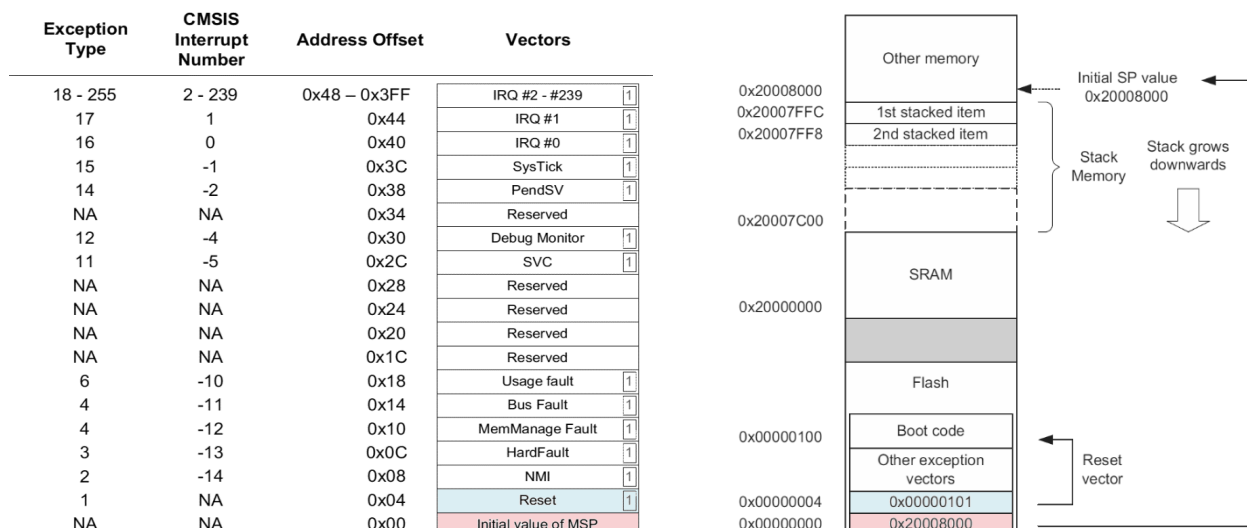
Lars Kamm, Jan Wendler

1 ARM CORTEX-M MEMORY ORGANIZATION

1.1 ARM Cortex-M Memory-Map

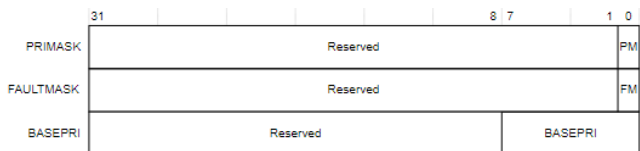
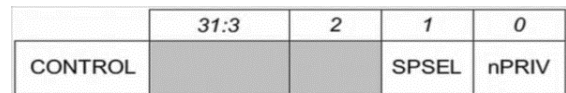
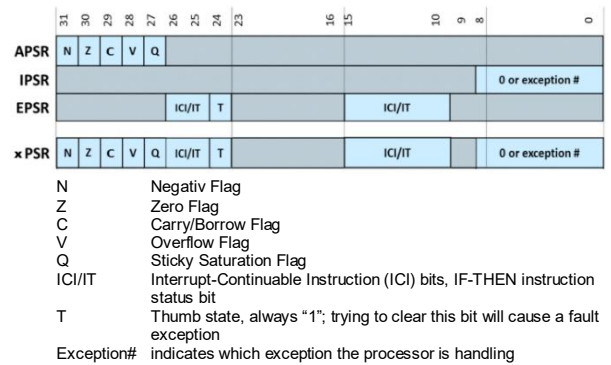


1.2 Vector-Table & Reset-Sequence

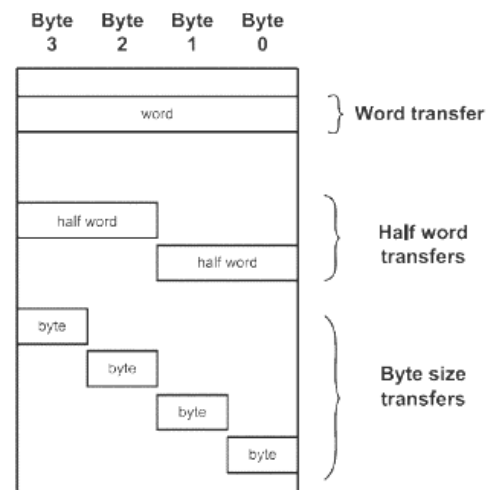
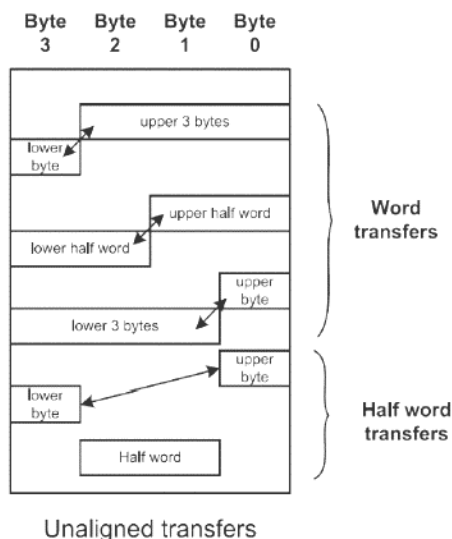


1.3 Register-Set & Program Status Register

Name	Functions (and banked registers)
R0	General purpose register
R1	General purpose register
R2	General purpose register
R3	General purpose register
R4	General purpose register
R5	General purpose register
R6	General purpose register
R7	General purpose register
R8	General purpose register
R9	General purpose register
R10	General purpose register
R11	General purpose register
R12	General purpose register
R13 (MSP)	Main Stack Pointer (MSP), Process Stack Pointer (PSP)
R14	Link Register (LR)
R15	Program Counter (PC)
xPSR	Program status registers
PRIMASK	Interrupt mask registers
FAULTMASK	Interrupt mask registers
BASEPRI	Control register
CONTROL	Control register



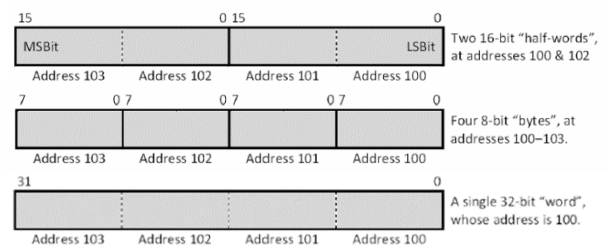
1.4 Alignment



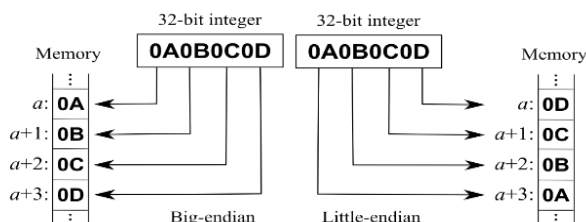
Word (32Bit): 0x0, 0x4, 0x8, 0xC

Halfword (16Bit): 0x0, 0x2, 0x4, 0x6, 0x8, 0xA, 0xC, 0xE

Byte (8Bit): 0x0, 0x1, ..., 0xF



1.5 Endianness



Endianness betrifft nur Zahlenformate die grösser als ein Byte sind (Halfword, Word, Doubleword).

Little-Endian: Least Significant Byte befindet sich an der tiefsten Adresse.

Big-Endian: Most Significant Byte befindet sich an der tiefsten Adresse.

1.6 Speichergrossen

Zur Angabe der Speicherkapazität als Anzahl von Bits, Bytes oder Codewörtern verwendet man in der Informatik in Anlehnung an die Physik die Bezeichnungen Kilo (K), Mega (M), Giga (G) und Tera (T). Diese beziehen sich auf das Binärzahlensystem:

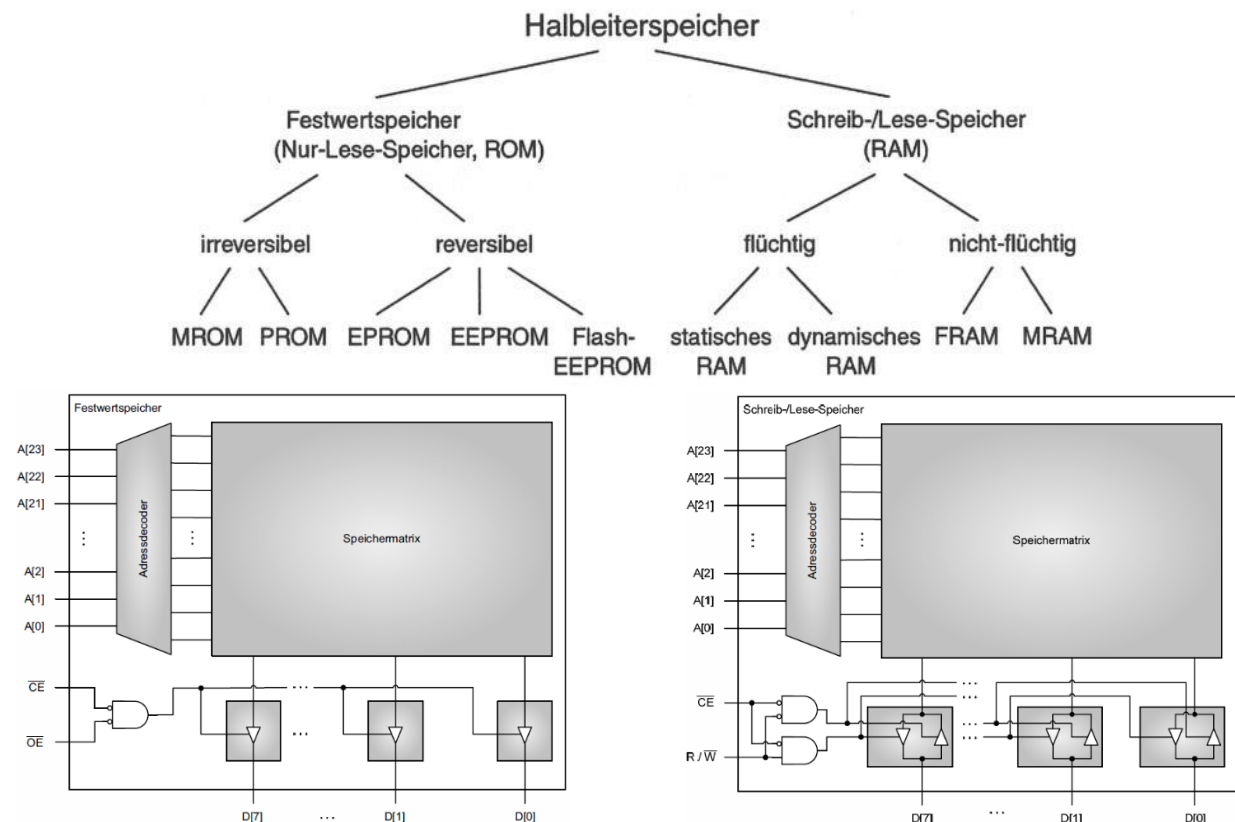
$$1K = 2^{10} = 1'024$$

$$1M = 2^{20} = 1'024K = 1'048'576$$

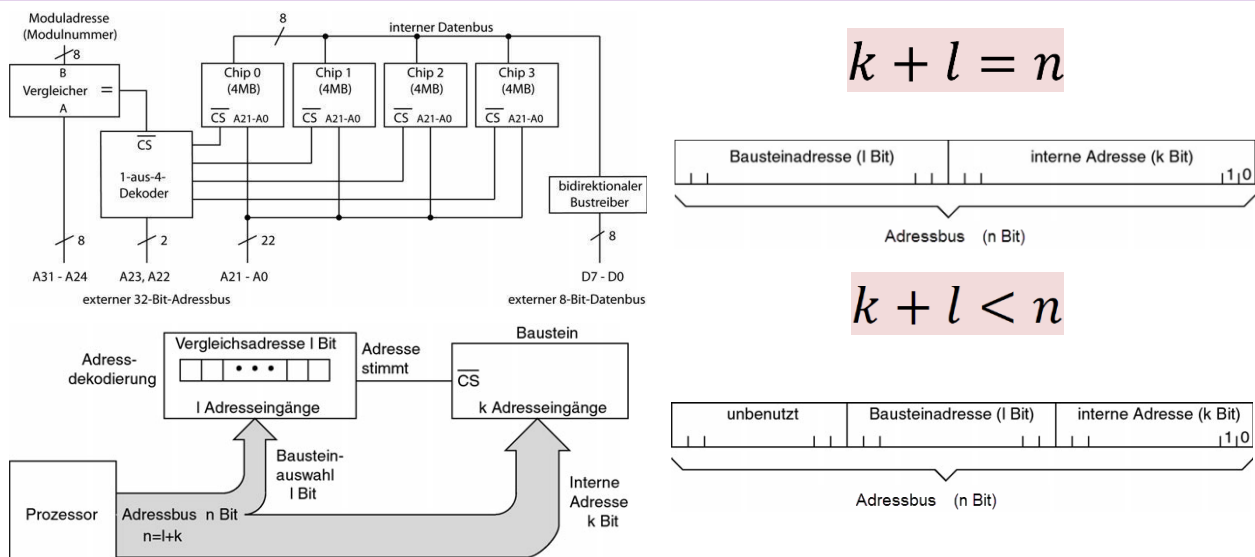
$$1G = 2^{30} = 1'024M = 1'048'576K = 1'073'741'824$$

$$1T = 2^{40} = 1'024G = 1'048'576M = 1'073'741'824K = 1'099'511'627'776$$

1.7 Speicherarten

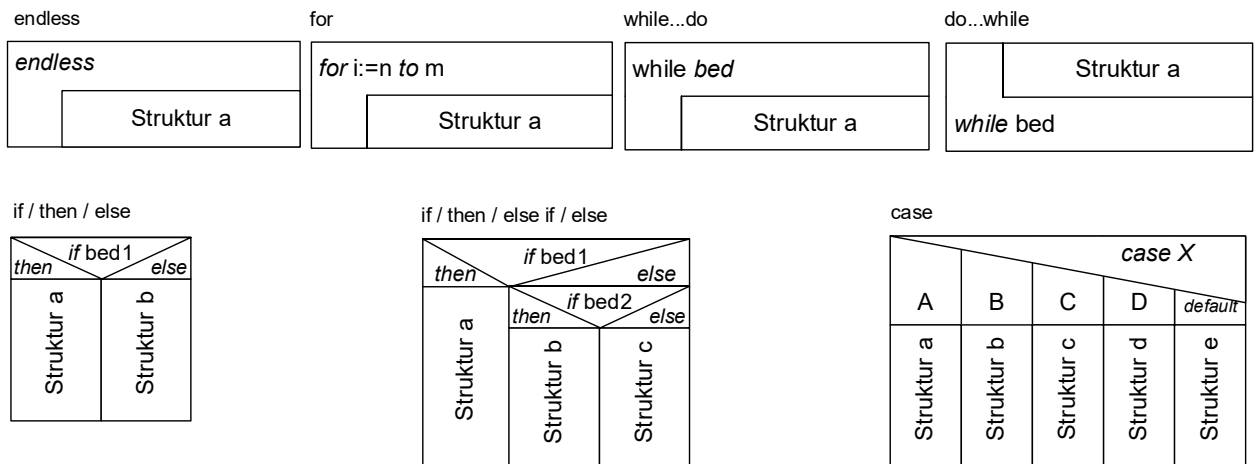


1.8 Adressdekodierung

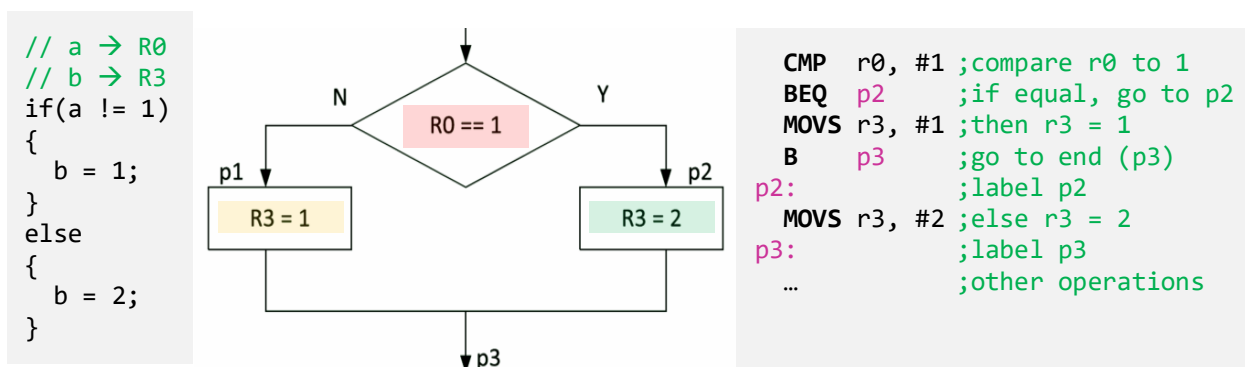


2 KONTROLL- & SELEKTIONSSTRUKTUREN

2.1 Nassi Schneidemann Diagramme



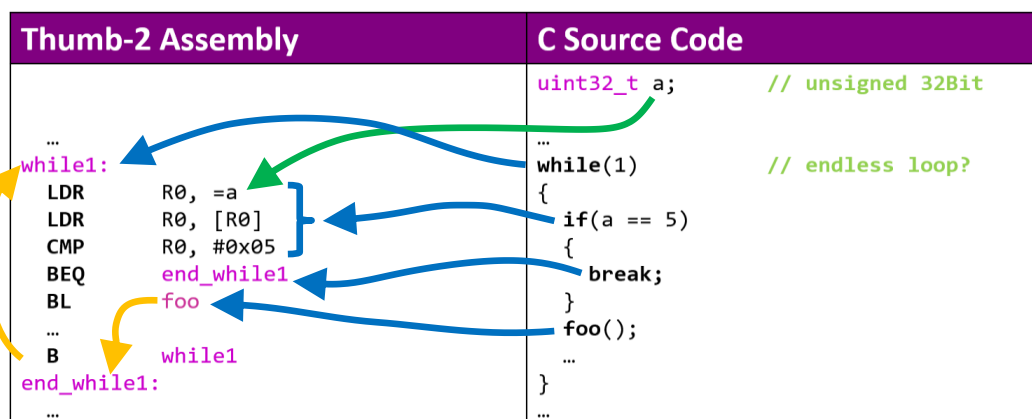
2.2 Entscheidungen treffen



In Assembler-Sprache ist eine Entscheidung praktisch immer ein 2-stufiger Ablauf

- Benötigte Flags ermitteln**
Vergleich (**Compare**) vornehmen, indem zwei Werte voneinander subtrahiert werden. Bei der Differenzbildung ist das Resultat nicht wichtig, sondern nur dessen Eigenschaften (**Flags**).
- Zugehörige Sprünge ausführen**
Im zweiten Schritt werden mit den Flags bedingte Sprünge (**B{cond}**) ausgeführt

2.3 Umsetzung von C/C++ Strukturen



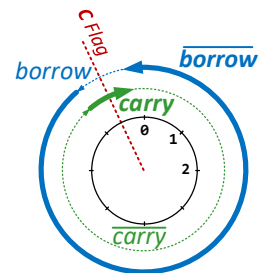
3 ARM CORTEX-M0(+) INSTRUCTION-SET

3.1 Symbole & Appendix

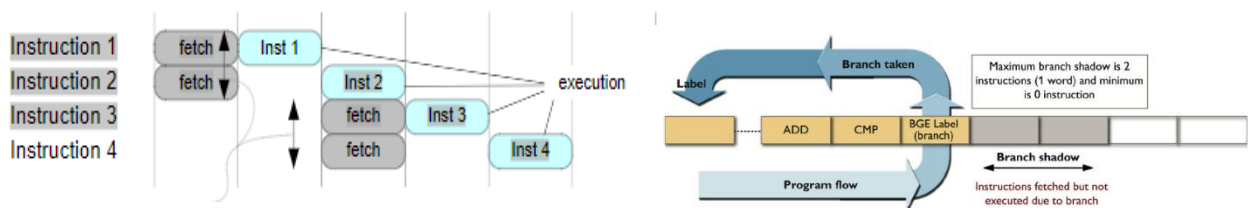
Symbols	Meaning
$R_d R_m R_n$	represent 32-bit registers
$\{R_d\}$	if R_d is present R_d is destination, otherwise R_n
$\{S\}$	if S is present, instruction will set condition codes
$\{cc\}$	optional logical condition, condition code suffix
$\#imm3$	any value in the range: 0...7
$\#imm5$	any value in the range: 0...31
$\#imm7$	any value in the range: 0..127
$\#imm8$	any value in the range: 0..255
$\#imm10$	any value in the range: 0..1023
$\#imm11$	any value in the range: 0..2047
label	any address within the ROM of the microcontroller; offset range relative to PC: B label -2 KB to +2 KB B<cc> label -256 bytes to +255 bytes BL label -16 MB to +16 MB
$\{reglist\}$	List of registers, sequence is not relevant

3.2 Bedingte Ausführung & Code-Zusätze {cc}

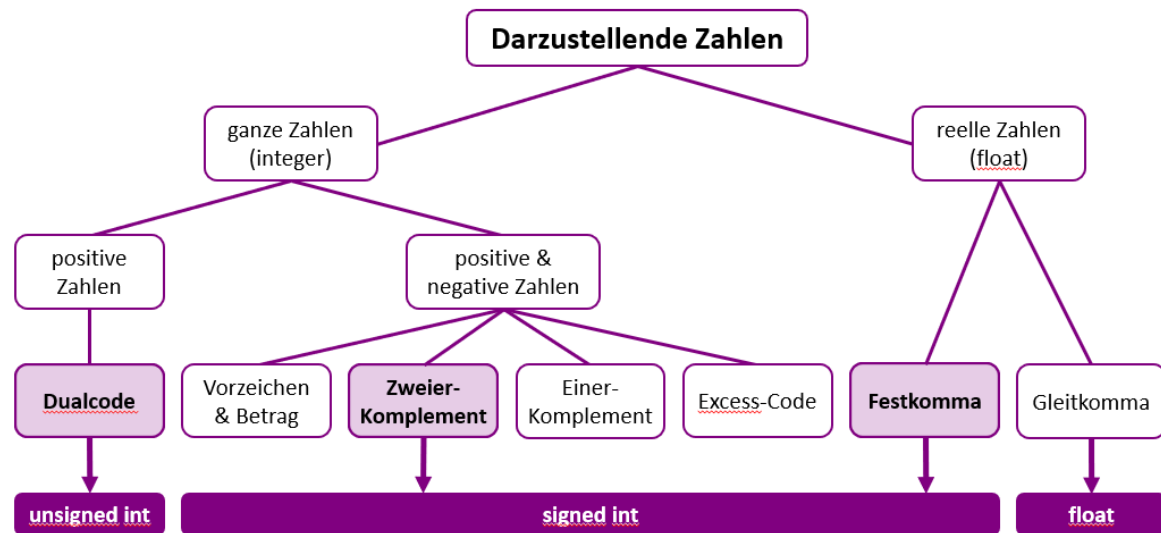
cc	Meaning	Flags (APSR) / Requirements	Binary	uint	int
EQ	Equal	Z = 1	0000	X ==	X ==
NE	Not equal	Z = 0	0001	X !=	X !=
CS or HS	Carry set, Unsigned ≥ carry OR borrow	C = 1	0010	X >=	
CC or LO	Carry clear, Unsigned < carry OR borrow	C = 0	0011	X <	
MI	Minus/negative	N = 1	0100		X -
PL	Positive or zero (non-negative)	N = 0	0101		X +
VS	Overflow	V = 1	0110		X
VC	No overflow	V = 0	0111		X
HI	Unsigned > ("Higher")	C = 1 && Z = 0	1000	X >	
LS	Unsigned ≤ ("Lower or Same")	C = 0 Z = 1	1001	X <=	
GE	Signed ≥ ("Greater than or Equal")	N = V	1010		X >=
LT	Signed < ("Less Than")	N ≠ V	1011		X <
GT	Signed > ("Greater Than")	Z = 0 && N = V	1100		X >
LE	Signed ≤ ("Less than or Equal")	Z = 1 N ≠ V	1101		X <=
AL	Always (unconditional)	any	1110		



3.3 Pipelining



4 ZAHLENSYSTEME



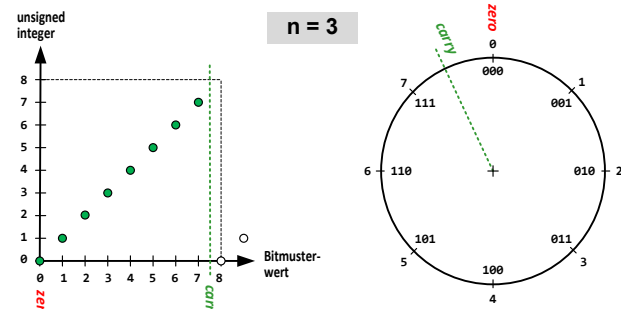
4.1 Vorzeichenlose Zahlen

Dualcode (unsigned int)

$$W = \sum_{i=0}^{n-1} d_i \cdot 2^i$$

$d_{n-1} \quad d_{n-2} \quad \dots \quad d_2 \quad d_1 \quad d_0$

Relevante Flags: Zero, Carry/Borrow



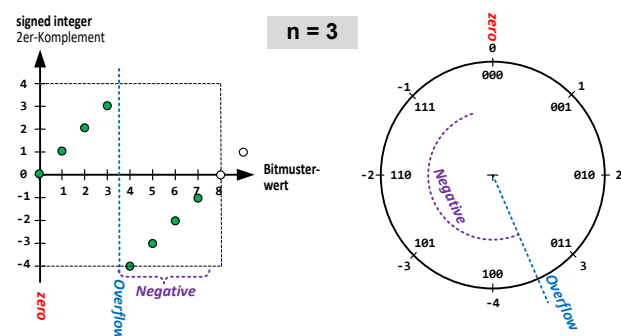
4.2 Vorzeichenbehaftete Zahlen

Zweierkomplement (signed int)

$$W = \left(\sum_{i=0}^{n-2} d_i \cdot 2^i \right) - d_{n-1} \cdot 2^{n-1}$$

$d_{n-1} \quad d_{n-2} \quad \dots \quad d_2 \quad d_1 \quad d_0$

Relevante Flag: Zero, Negative, Overflow



4.3 Zweierkomplement-Negierung

Vorgehensweise:

- 1) Vorzeichenbehaftete Zahl in Binärcode darstellen
- 2) Alle Bits einzeln invertieren
- 3) Binäre Addition mit dem Wert +1
- 4) Resultat entspricht der negierten Vorzeichenbehaftete Zahl

Beispiel: $-3_{10} = 101_2$
 Invertieren: 010_2
 +1: 001_2

 011_2
 =====
 Resultat: $+3_{10}$

4.4 Festkomma-Zahlen

Vorzeichenlose Fixed-Point-Zahlen

Allgemeine Formel (Dualcode als Basis)

$$W = \sum_{i=-m}^{n-1} d_i \cdot 2^i$$

$$d_{n-1} \ d_{n-2} \ \dots \ d_1 \ d_0 \ . \ d_{-1} \ d_{-2} \ d_{-m}$$

Wertebereich: $0.0 \dots 2^n - 2^{-m}$

Schrittweite: $\Delta X = 2^{-m}$

Max. Diskretisierungsfehler: $\frac{\Delta X}{2} = 2^{-(m+1)}$

Vorzeichenbehaftete Fixed-Point-Zahlen

Allgemeine Formel (2er-Komplement als Basis)

$$W = \left(\sum_{i=-m}^{n-2} d_i \cdot 2^i \right) - d_{n-1} \cdot 2^{n-1}$$

$$d_{n-1} \ d_{n-2} \ \dots \ d_1 \ d_0 \ . \ d_{-1} \ d_{-2} \ d_{-m}$$

Wertebereich: $-2^{n-1} \dots 2^{n-1} - 2^{-m}$

Schrittweite: $\Delta X = 2^{-m}$

Max. Diskretisierungsfehler: $\frac{\Delta X}{2} = 2^{-(m+1)}$

4.5 IQ-Zahlenformate

Integer/Quotient-Darstellung

Signed Fixed-Point mit $n = I$ und $m = Q$.

$$W = -d_{I-1} \cdot 2^{I-1} + \sum_{j=0}^{I-2} d_j \cdot 2^j + \sum_{i=1}^Q d_{-i} \cdot 2^{-i}$$

$$d_{I-1} \ d_{I-2} \ \dots \ d_1 \ d_0 \ . \ d_{-1} \ d_{-2} \ d_{-Q}$$

Fraktionale Zahlen (oder Spezialfall I1Qx)

$$W = -d_0 + \sum_{i=1}^Q d_{-i} \cdot 2^{-i}$$

$$d_0 \ . \ d_{-1} \ d_{-2} \ \dots \ d_{-Q}$$

Wertebereich: $-1.0 \leq W < +1.0$

Vorzeichen: Koeffizient d_0

Genauigkeit: $2^{-(Q)}$

4.6 Zahlenkreise

Addition: Zahlenvektoren hintereinander reihen (Anfang auf Ende)

Subtraktion: Zahlenvektoren stumpf gegeneinanderstellen (Ende auf Ende)

Unsigned Integer

Relevante Flags: Zero, Carry/Borrow

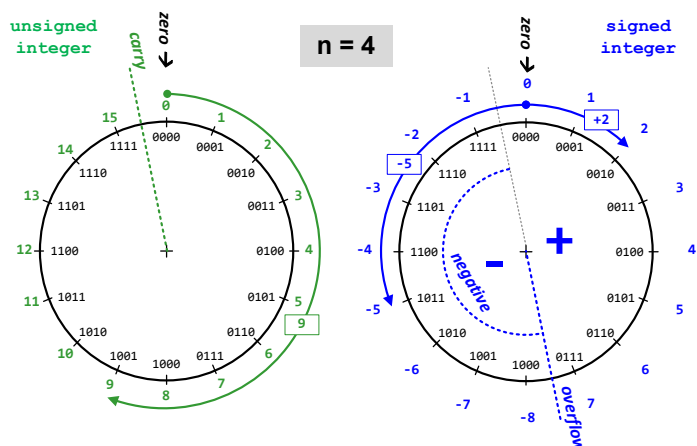
Zahlenvektoren immer Uhrzeigersinn

Signed Integer

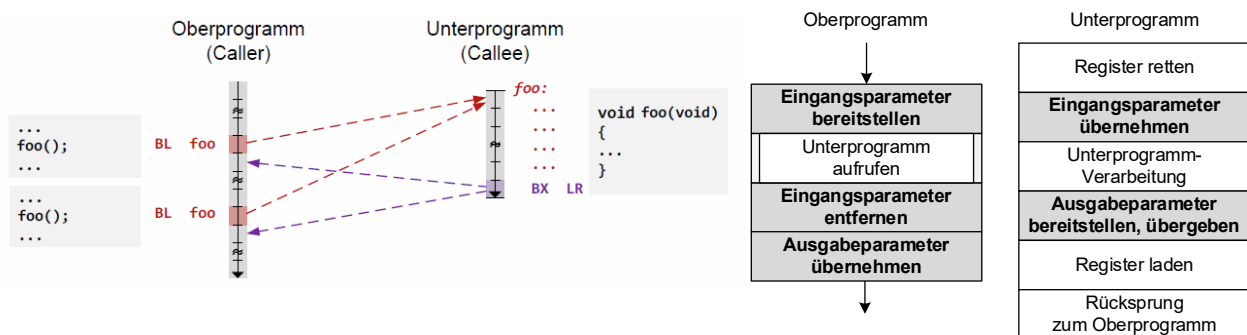
Relevante Flags: Negative, Overflow, Zero

positiv → Zahlenvektor Uhrzeigersinn

negativ → Zahlenvektor Gegenuhrzeiger



5 SUBROUTINEN



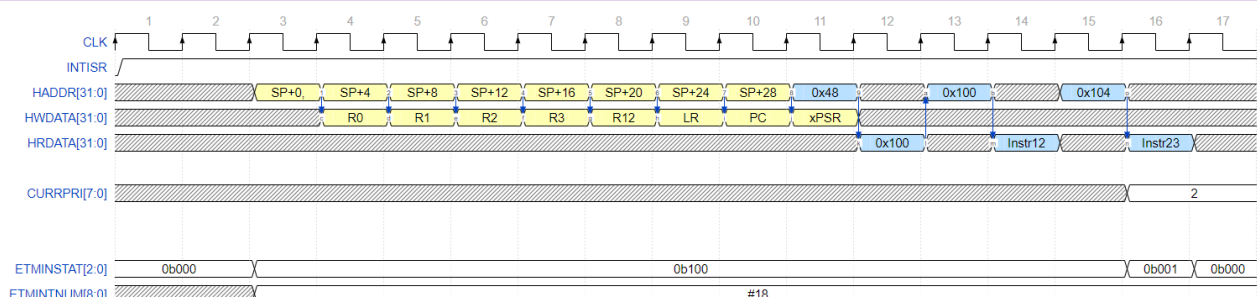
Regeln zum ARM Architecture Procedure Call Standard (AAPCS)

- Die Register **R0-R3** werden als Parameter verwendet.
- Weitere Parameter werden auf den **Stack** gelegt.
- Der Rückgabewert ($\leq 32\text{Bit}$) wird in Register **R0** übertragen.
- Ein 64Bit-Rückgabewert wird in den Registern **R0** und **R1** übertragen.
- Funktionen müssen die Registerinhalte **R4-R11** während der Ausführung sichern und rekonstruieren.
- Die Register **R0-R3** und **R12** dürf(t)en in der Subroutine ohne Sicherung verändert werden.
- Es wird stets ein **8-Byte Alignment** auf dem Stack eingehalten.

Register	Verwendungszweck	Scratch	Sichern
R0	Erster Funktionsparameter, Return-Wert (8-/16-/32-Bit)	Ja	
R1	Zweiter Funktionsparameter, Upper Word Return-Wert (64-Bit)	Ja	
R2, R3	Dritter und vierter Funktionsparameter	Ja	
R4-R7	Registervariablen (temporär)	Ja	Ja
R8-R11	Compilerabhängige Verwendung	Ja	Ja
R12	Funktionsinternes temporäres Speicherregister für Sprünge	Ja	
R13	Stack-Pointer (SP, Top-of-Stack)	Diese Register werden für spezielle Zwecke verwendet	
R14	Link Register (LR)		
R15	Programm Counter (PC)		

6 INTERRUPTS & EXCEPTIONS

6.1 Vector Fetch & Stacking



Request:	Pipeline «Execute» noch ausführen und der Pending Status setzen
Stacking & Vector-Fetch:	Register auf Stack sichern und Sprung via Vektortabelle ausführen und Active Status setzen.
ISR Handler:	Benutzercode ausführen und am Ende den Active Status zurücksetzen
Unstacking:	Register wieder herstellen, Stack abbauen und Rücksprung mittels LR

7 PERSONAL NOTES

